Machine Learning Operations (AIMLCZG523)

Assignment 1

Heart Disease Risk Prediction:

An End-to-End MLOps Pipeline from Data to a Monitored API

Student: Komal Matlawala

BITS ID: 2025AE05243

Programme: M.Tech in AI & ML, BITS Pilani (WILP)

Code repository: https://github.com/Kmatlawala/heart-disease-mlops

Deployed API: Local Kubernetes (Docker Desktop); access steps in Section 10

Submission date: October 2026

Contents

1. Introduction
2. Setup and Installation
3. Data Acquisition and Cleaning
4. Exploratory Data Analysis
5. Feature Engineering and Model Development
6. Experiment Tracking with MLflow
7. Model Packaging and Reproducibility
8. CI/CD Pipeline and Automated Testing
9. Model Containerization
10. Production Deployment on Kubernetes
11. Monitoring and Logging
12. Architecture
13. Conclusion and Future Work

1. Introduction

Heart disease is a leading cause of death worldwide. Spotting high-risk patients early from routine measurements can help doctors decide who needs further tests. This project builds a classifier that predicts whether a patient has heart disease from 13 clinical measurements, and turns it into a production-style service: tested, containerised, deployed on Kubernetes and monitored.

The focus of the assignment is not only the model but the engineering around it. Every step is automated and reproducible: one command downloads the data, another trains and tracks the models, the CI pipeline re-runs everything on each push, and the same Docker image runs on a laptop and in the cluster.

1.1 Summary of results

- **Best model:** Logistic Regression (C = 1, L2), chosen by 5-fold cross-validated ROC-AUC (0.908 ± 0.018).
- **Held-out test set (61 patients):** ROC-AUC 0.960, accuracy 0.869, recall 0.929, precision 0.813.
- **Serving:** FastAPI service with /predict, /predict/batch, /health, /model-info and /metrics endpoints, packaged in a slim non-root Docker image.
- **Automation:** GitHub Actions pipeline with lint → unit tests → training with a quality gate → Docker build and smoke test → image push.
- **Operations:** Kubernetes Deployment with 2 replicas, health probes and a LoadBalancer Service; Prometheus + Grafana dashboard and structured JSON request logs.

2. Setup and Installation

The project needs Python 3.12 or newer, Git and Docker. Kubernetes can be Docker Desktop's built-in cluster or Minikube. All library versions are pinned in requirements.txt, so a clean install reproduces the exact environment.

```
git clone <repo-url> && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\
activate
pip install -r requirements.txt
export PYTHONPATH=src:.

python -m heart.data     # download + clean
python -m heart.eda      # EDA charts
python -m heart.train    # tune, track, save model
python -m pytest -v      # unit tests
uvicorn api.main:app --port 8000   # serve locally
```

The repository layout separates concerns clearly: src/heart holds the reusable Python package (data, features, training, prediction), api/ holds the web service, tests/ holds unit tests, k8s/ and monitoring/ hold deployment and observability configuration, and notebooks/ holds executed EDA, training and inference notebooks that call the same package code.

3. Data Acquisition and Cleaning

Source. The UCI Heart Disease dataset, Cleveland subset (303 patients, 13 features). The script heart/data.py downloads processed.cleveland.data from the UCI repository. If UCI cannot be reached (it is sometimes slow or blocked on corporate networks), it falls back to a public mirror of the same 303 records and converts its text labels to the UCI numeric codes, so both paths produce an identical table.

Cleaning steps. The cleaning function is deterministic and unit-tested:

- '?' markers are read as missing values and every column is converted to a number.
- The original target 'num' (0 to 4, severity) is turned into a binary target: 0 = no disease, 1 = disease (num > 0).
- Values outside medically possible ranges (for example cholesterol above 700 mg/dl) or unknown category codes are set to missing, so bad input can never silently reach the model.
- Exact duplicate rows and rows without a target are removed.

Missing values. Only ca (4 rows) and thal (2 rows) are missing. They are deliberately not filled in the CSV. Instead, the model pipeline fills them (median for numbers, most frequent value for categories). This guarantees that a missing value at prediction time is handled exactly as in training.

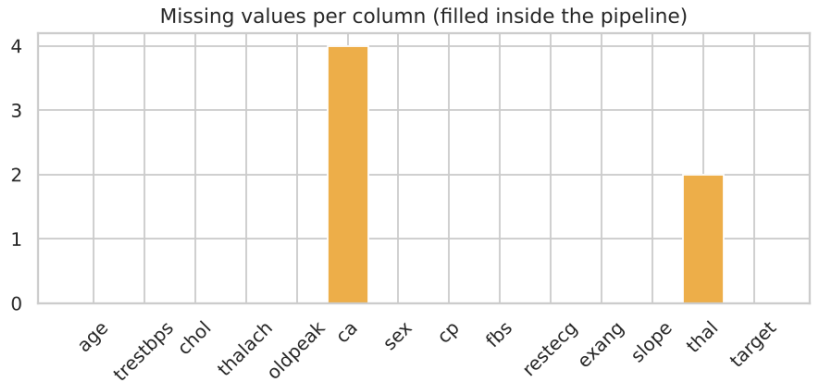


Figure 1. Missing values per column after cleaning

4. Exploratory Data Analysis

All charts are produced by heart/eda.py, which the notebook 01_eda.ipynb and the CI pipeline both call, so the figures are always regenerated from the current data.

4.1 Class balance

There are 164 patients without disease (54%) and 139 with disease (46%). The classes are close to balanced, so no resampling is needed. Stratified splits are still used everywhere so each fold keeps this ratio.

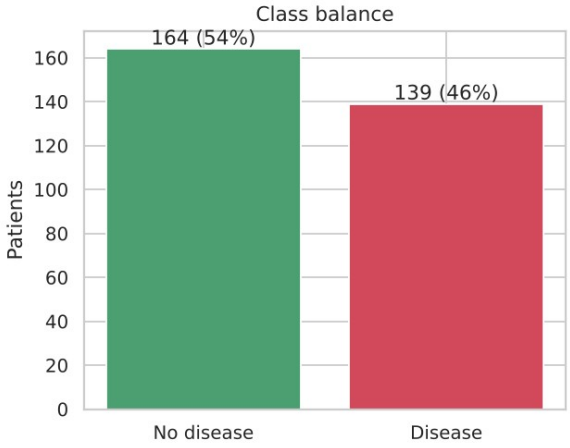


Figure 2. Class balance

4.2 Numeric features

Patients with disease tend to have a lower maximum heart rate (thalach), a higher ST depression (oldpeak) and more coloured vessels (ca). Age is slightly higher in the disease group. Cholesterol and

resting blood pressure overlap heavily between the groups, so they are weak predictors on their own.

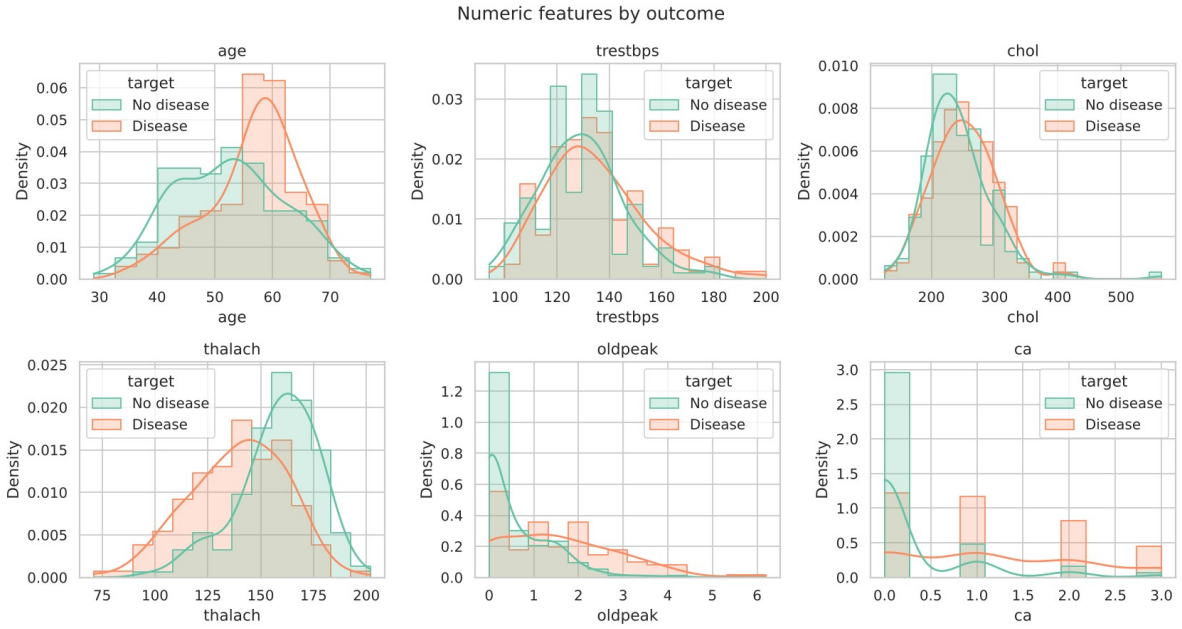


Figure 3. Distribution of numeric features by outcome

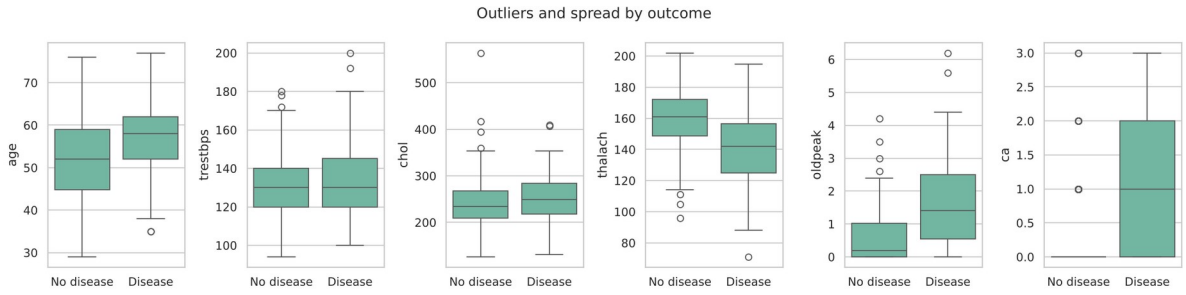


Figure 4. Spread and outliers by outcome

4.3 Categorical features

Asymptomatic chest pain (cp = 4), exercise-induced angina (exang = 1) and a reversible thalassemia defect (thal = 7) all show disease rates above 70%. Fasting blood sugar (fbs) makes almost no difference. Because cp, thal, restecg and slope are codes rather than amounts, they are one-hot encoded instead of treated as numbers.

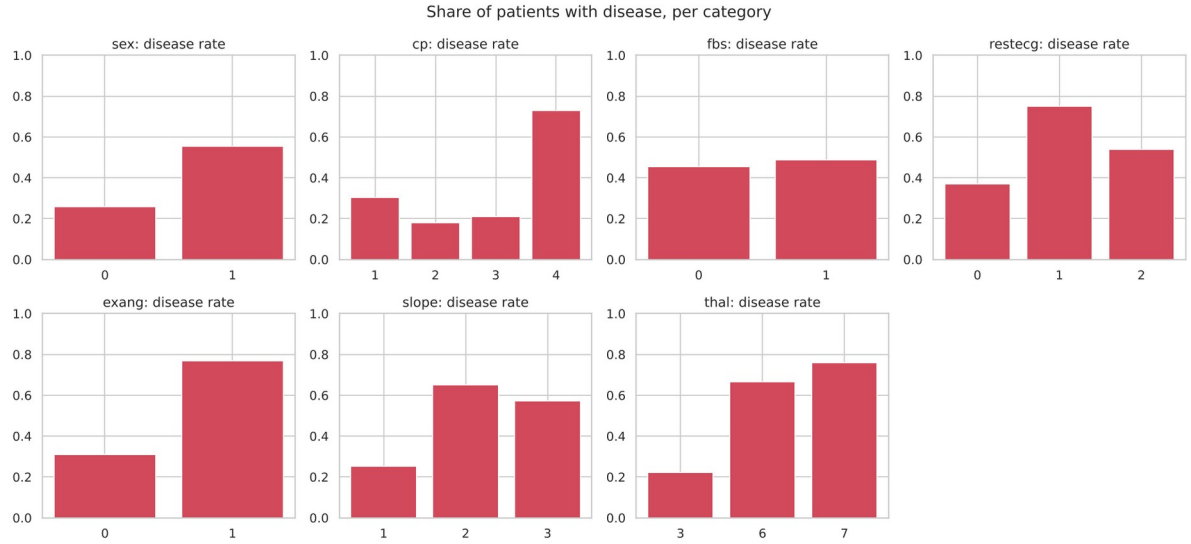


Figure 5. Share of patients with disease in each category

4.4 Correlation

The features most correlated with the target are thal (0.53), ca (0.46), exang (0.43), oldpeak (0.42), thalach (−0.42) and cp (0.41). No pair of input features is so strongly correlated that one should be dropped, so all 13 features are kept.

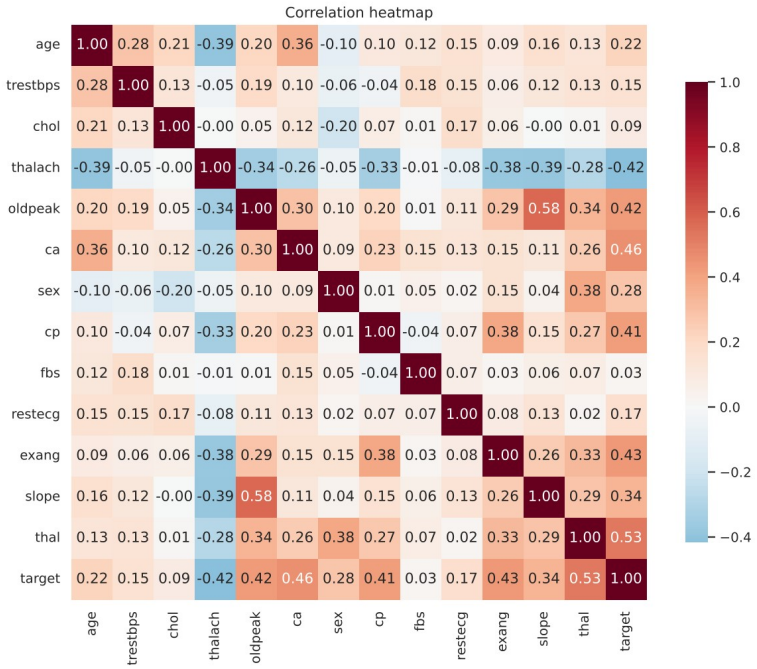


Figure 6. Correlation heatmap

5. Feature Engineering and Model Development

5.1 Preprocessing pipeline

Preprocessing is a single scikit-learn ColumnTransformer combined with the model in one Pipeline object:

- **Numeric** (age, trestbps, chol, thalach, oldpeak, ca): median imputation, then standard scaling.

- **Categorical** (sex, cp, fbs, restecg, exang, slope, thal): most-frequent imputation, then one-hot encoding (binary columns kept as one column; unknown codes at prediction time are ignored safely).

Because the scaler and encoder are fitted inside each cross-validation fold, no information from validation data leaks into training.

5.2 Models and tuning

The data is split 80/20 (stratified, fixed random seed). Three models are tuned with GridSearchCV using 5-fold stratified cross-validation and ROC-AUC as the score. ROC-AUC was chosen because it measures how well the model ranks risky patients regardless of the 0.5 cut-off.

Model	Search space	Best setting
Logistic Regression	C ∈ {0.01, 0.1, 1, 10}; penalty ∈ {L1, L2}	C = 1.0, L2
Random Forest	trees ∈ {200, 400}; max depth ∈ {3, 5, none}; min leaf ∈ {1, 5}	200 trees, depth 5, min leaf 5
Gradient Boosting	trees ∈ {100, 200}; learning rate ∈ {0.05, 0.1}; depth ∈ {2, 3}	100 trees, rate 0.05, depth 3

5.3 Evaluation

Model	CV ROC-AUC	CV accuracy	CV recall	Test ROC-AUC	Test acc.	Test recall
Logistic Regression	**0.908 ± 0.018**	**0.843**	**0.774**	**0.960**	**0.869**	**0.929**
Random Forest	0.902 ± 0.022	0.826	0.764	0.955	0.869	0.929
Gradient Boosting	0.874 ± 0.028	0.789	0.783	0.946	0.885	0.929

Model selection. Logistic Regression was selected because it has the highest mean CV ROC-AUC and the lowest variance across folds. The test set was only used after the choice was made, so its scores are an honest estimate. Gradient Boosting has slightly higher test accuracy, but on 61 test patients this is a difference of one patient and is not significant; its CV scores are clearly lower. Logistic Regression also has practical advantages: its coefficients are easy to explain to clinicians, and it is tiny and fast to serve.

Recall matters. In a screening setting, missing a sick patient is worse than a false alarm. The selected model catches 26 of the 28 disease cases in the test set (recall 0.929). The decision threshold (0.5) is a parameter in predict.py and could be lowered if even higher recall were needed.

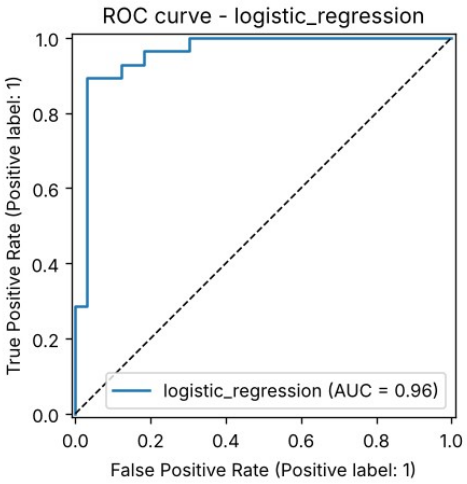


Figure 7. ROC curve of the selected model on the test set

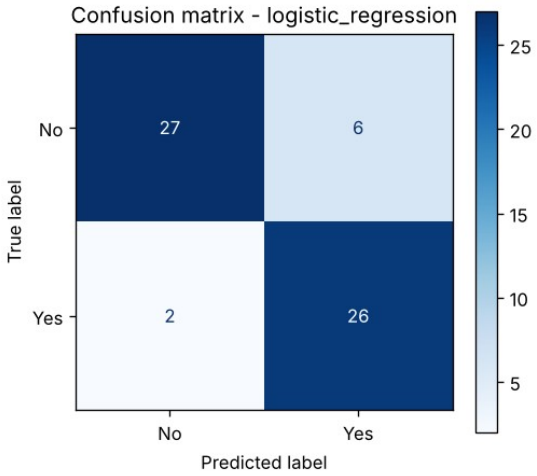


Figure 8. Confusion matrix of the selected model

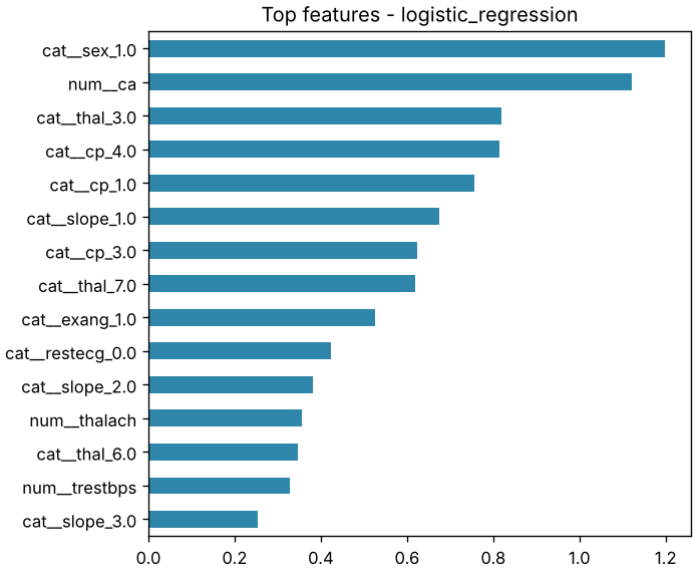


Figure 9. Largest coefficients of the selected model (absolute value)

6. Experiment Tracking with MLflow

MLflow is integrated in heart/train.py with a SQLite backend (mlflow.db), so it works without any server. Each training run creates the following in the experiment 'heart-disease-classifier':

- **One parent run per model** with the best parameters, all CV metrics (mean and standard deviation of accuracy, precision, recall, F1, ROC-AUC), all test metrics, tags, plots (ROC curve, confusion matrix, feature importance), the full grid-search results CSV, and the fitted pipeline with its input signature and an input example.
- **One child run per grid candidate** (8 + 12 + 8 = 28 runs) with its parameters and CV ROC-AUC, so the whole tuning process can be compared in the MLflow UI.
- **Model registry:** the winning pipeline is registered as 'heart-disease-model', giving it a version number.

The UI is opened with: mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000.

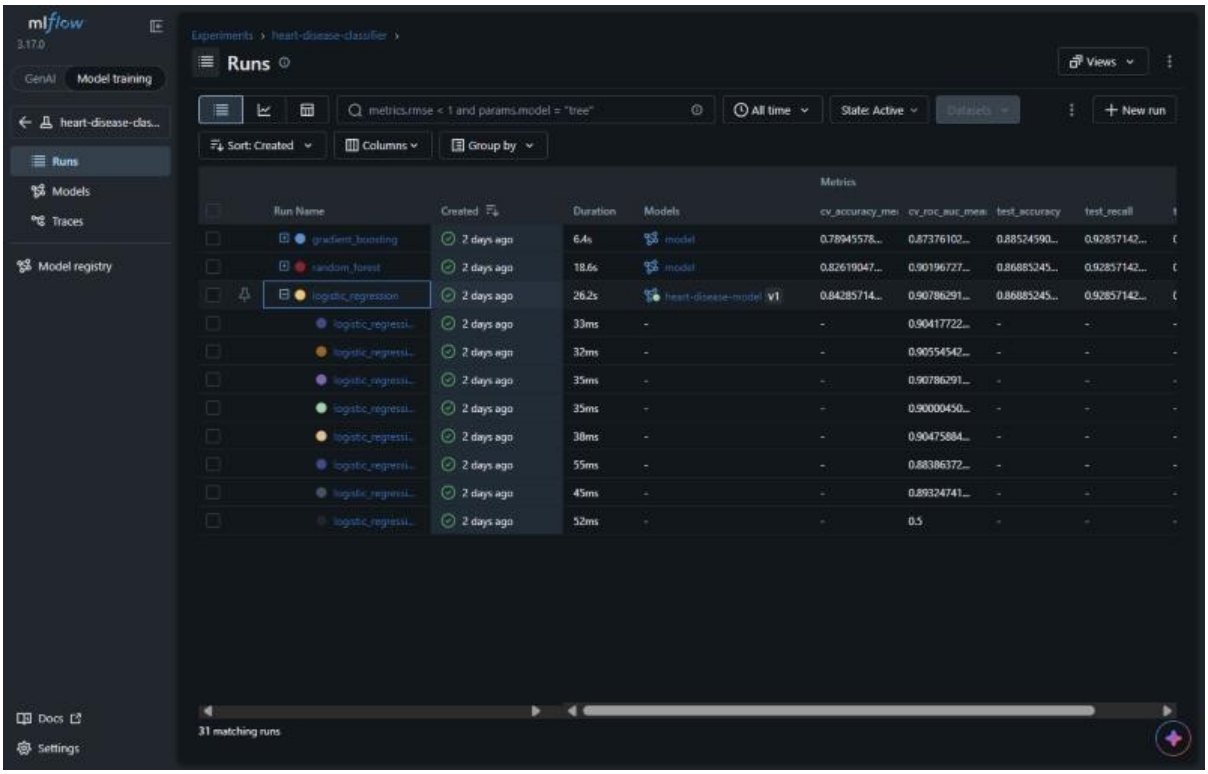


Figure 11. MLflow runs: 3 tuned models with CV/test metrics, and the 8 grid-search child runs of logistic_regression

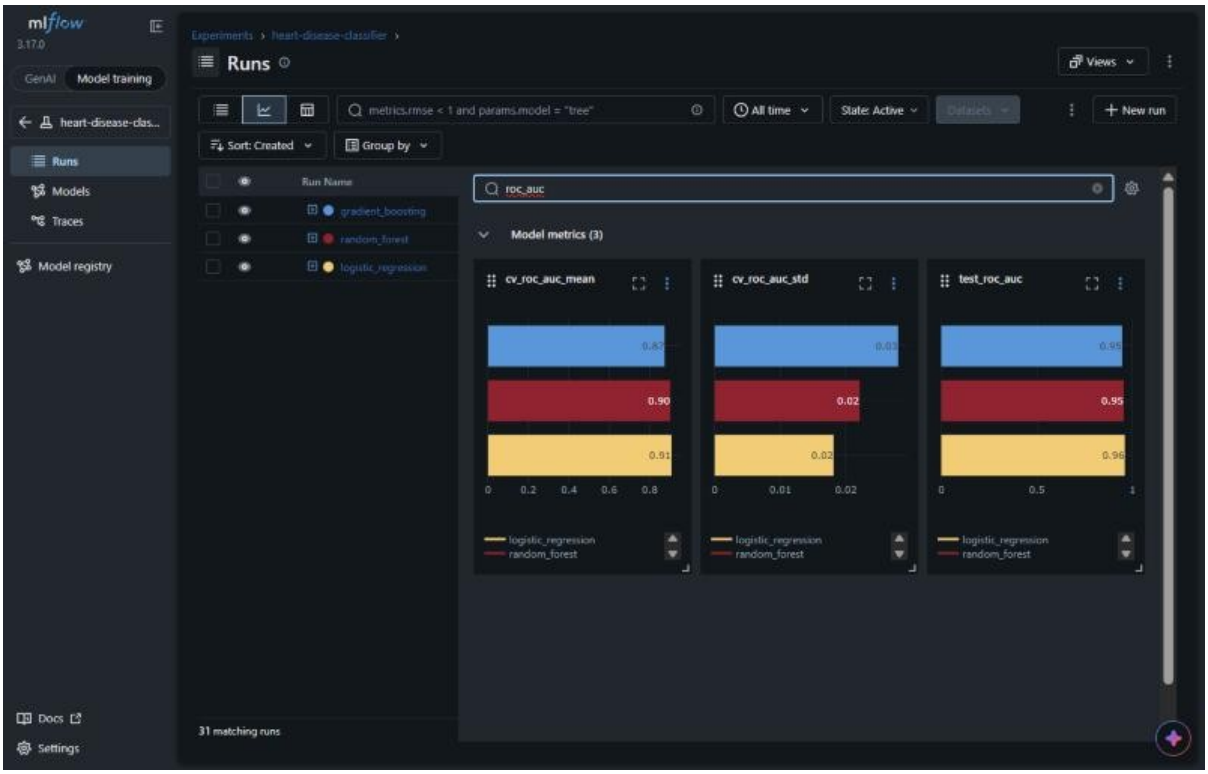


Figure 12. MLflow chart view comparing the models on CV ROC-AUC (mean, std) and test ROC-AUC

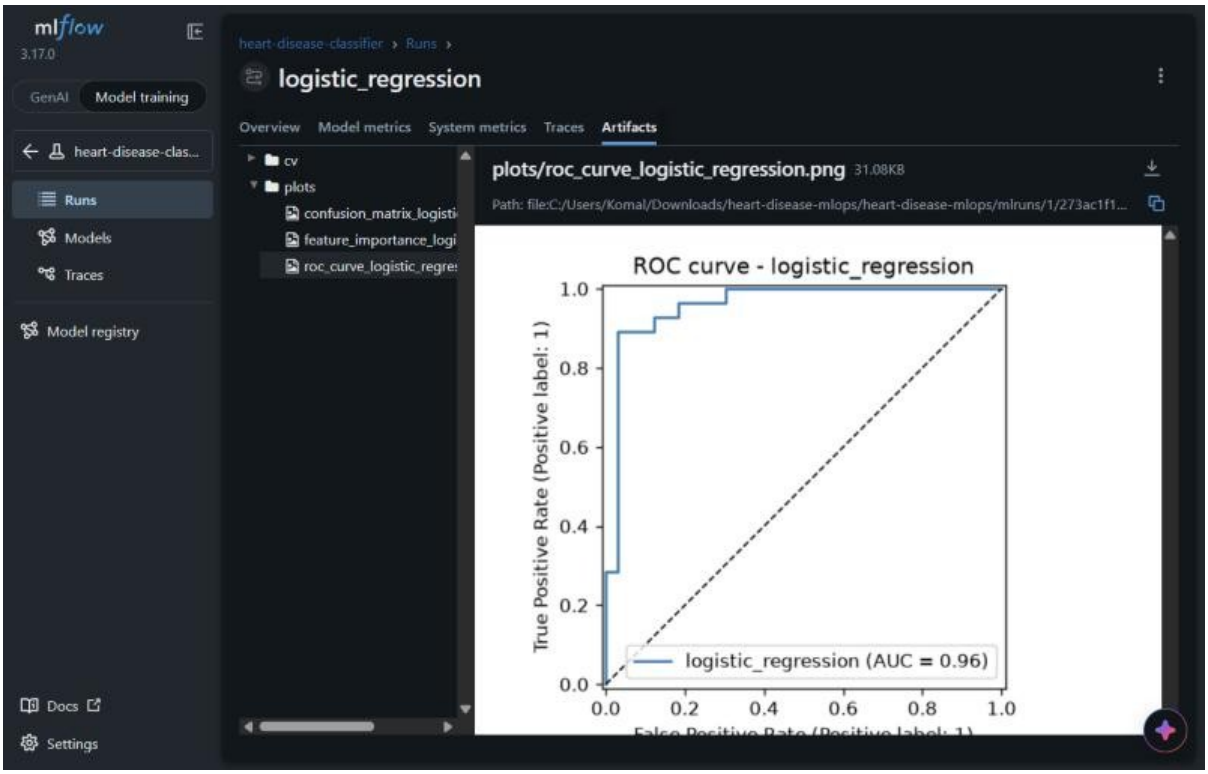


Figure 13. Artifacts logged for the logistic_regression run (ROC curve, confusion matrix, feature importance, CV results)

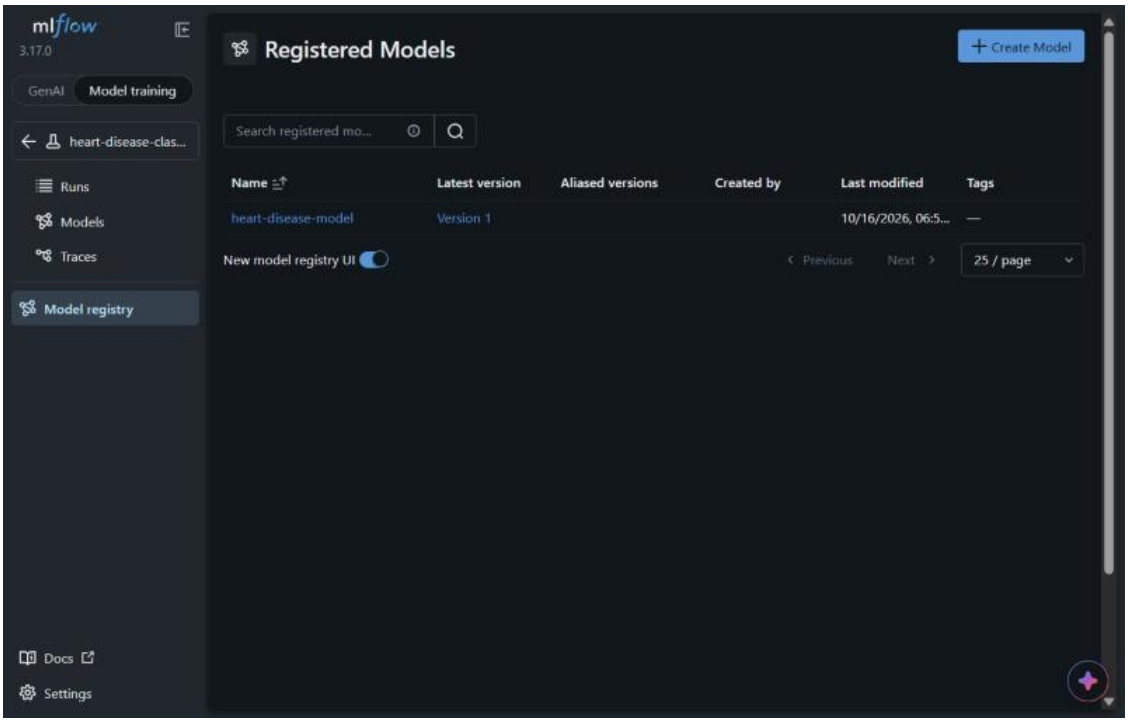


Figure 14. The selected model registered as heart-disease-model, version 1

7. Model Packaging and Reproducibility

- **One file, one pipeline:** models/model.joblib stores preprocessing and model together, so serving cannot drift from training. The same model is also saved in MLflow format with its signature.
- **Metadata:** models/model_meta.json records model name, parameters, all metrics, MLflow run ID, feature list, training time, and the Python and scikit-learn versions.
- **Pinned dependencies:** requirements.txt (full environment) and requirements-serve.txt (only what serving needs) pin the same versions, so the model is always loaded by the scikit-learn version that created it.
- **Determinism:** fixed random seed for the split, CV folds and models. Two clean runs on different machines produced identical metrics.
- **Clean-setup check:** the pipeline was verified from scratch in a fresh virtual environment using only requirements.txt: download → EDA → train → lint → tests → serve.

The project was also cloned from GitHub and run on the BITS Prayogshala lab machine (Linux, Python 3.12) using only requirements.txt; data download, training and all unit tests ran without changes.

```
(base) s2025ae05243@ip-10-3-18-180:~/workspace/MLOps/heart-disease-mlops$ python -m pytest -q
.....................                                                    [100%]
=============================== warnings summary ===============================
../../../../cloud/miniconda3/lib/python3.12/site-packages/fastapi/testclient.py:1
  /home/cloud/miniconda3/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` i
s deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

tests/test_model.py::test_unknown_category_does_not_crash
  /home/cloud/miniconda3/lib/python3.12/site-packages/sklearn/preprocessing/_encoders.py:262: UserWarning: Found unknown categories in columns [3] duri
ng transform. These unknown categories will be encoded as all zeros
    warnings.warn(msg, UserWarning)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
21 passed, 2 warnings in 4.17s
(base) s2025ae05243@ip-10-3-18-180:~/workspace/MLOps/heart-disease-mlops$
```

Figure 28. Clean setup on the Prayogshala lab machine: all 21 unit tests pass

8. CI/CD Pipeline and Automated Testing

8.1 Unit tests

The tests/ folder contains 21 pytest tests that use synthetic data, so they run in seconds and need no network:

- **Data (8 tests):** binary target, column order, '?' handling, numeric types, invalid values set to missing, duplicates removed, parsing of both the UCI and mirror formats.
- **Model (7 tests):** preprocessing leaves no missing values and scales correctly, unknown categories do not crash, stratified split, all metrics in [0, 1], feature importance for two model types, output format, save/reload gives identical predictions.
- **API (6 tests):** /health, /predict output, rejection of missing and out-of-range fields (HTTP 422), batch prediction, and that /metrics counts predictions.

8.2 GitHub Actions workflow

The workflow .github/workflows/ci.yml runs on every push, pull request and on demand. Each job depends on the previous one, so the pipeline stops at the first failure and the logs show the cause:

Job	What it does	Artifacts saved
1. lint	flake8 on src, api and tests	flake8 report
2. test	pytest with coverage	JUnit XML, coverage XML
3. train	download data, EDA, train with MLflow, quality gate: test ROC-AUC ≥ 0.80	model, metadata, figures, training log, MLflow runs
4. docker	build image with the new model, start it, call /health and /predict, check the response; push to GHCR on main	prediction JSON, container log

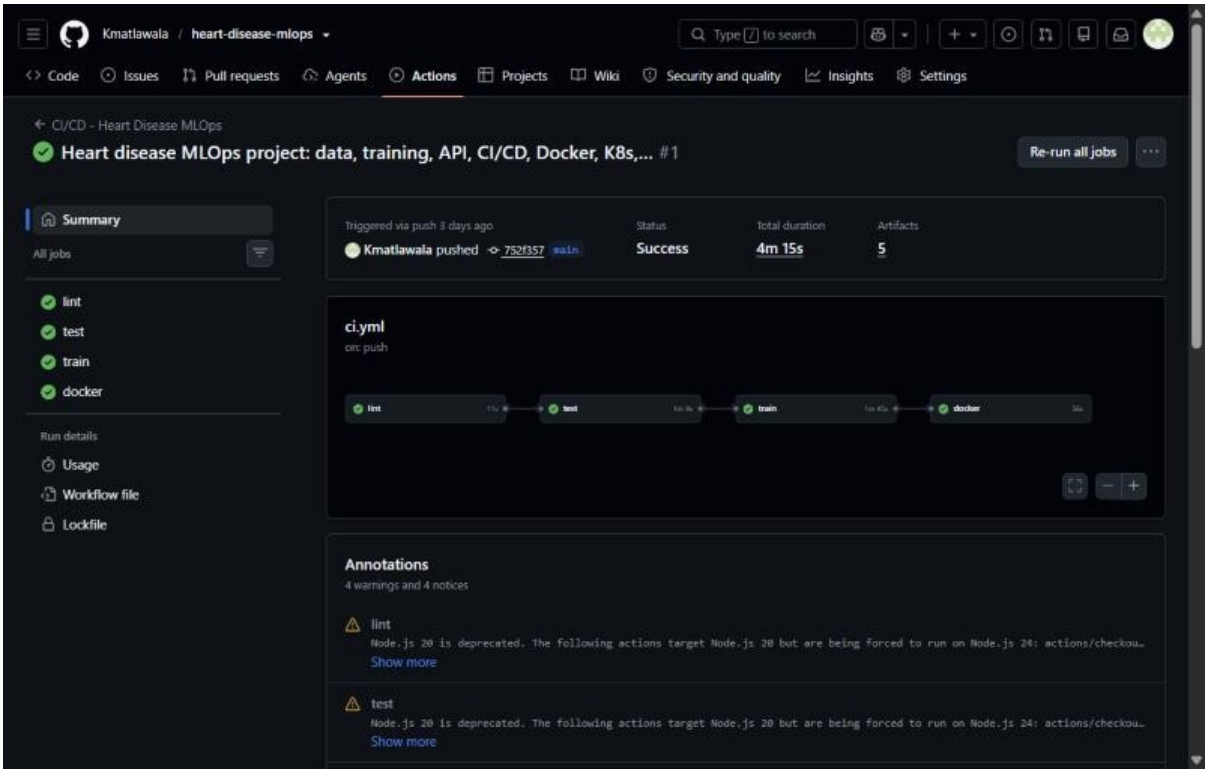


Figure 15. GitHub Actions run: lint → test → train → docker, all successful in 4 min 15 s

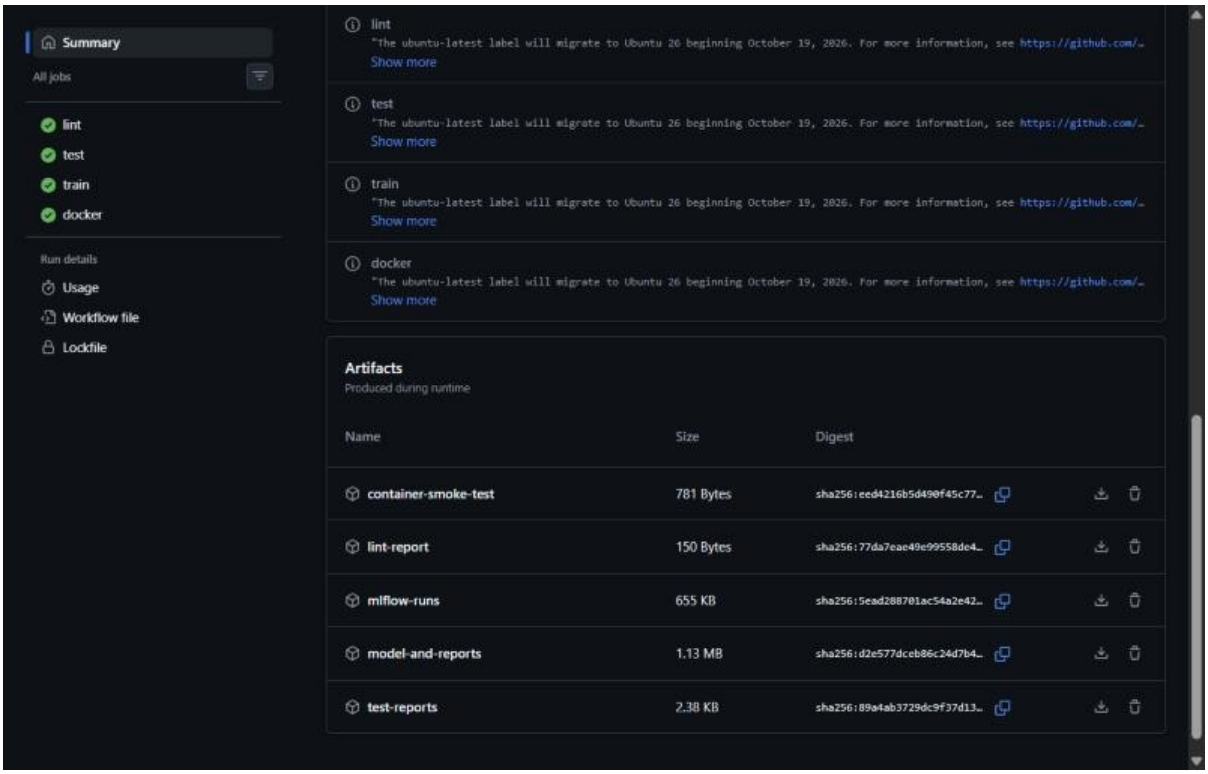


Figure 16. Artifacts uploaded by the run: lint report, test reports, MLflow runs, model + reports, container smoke test

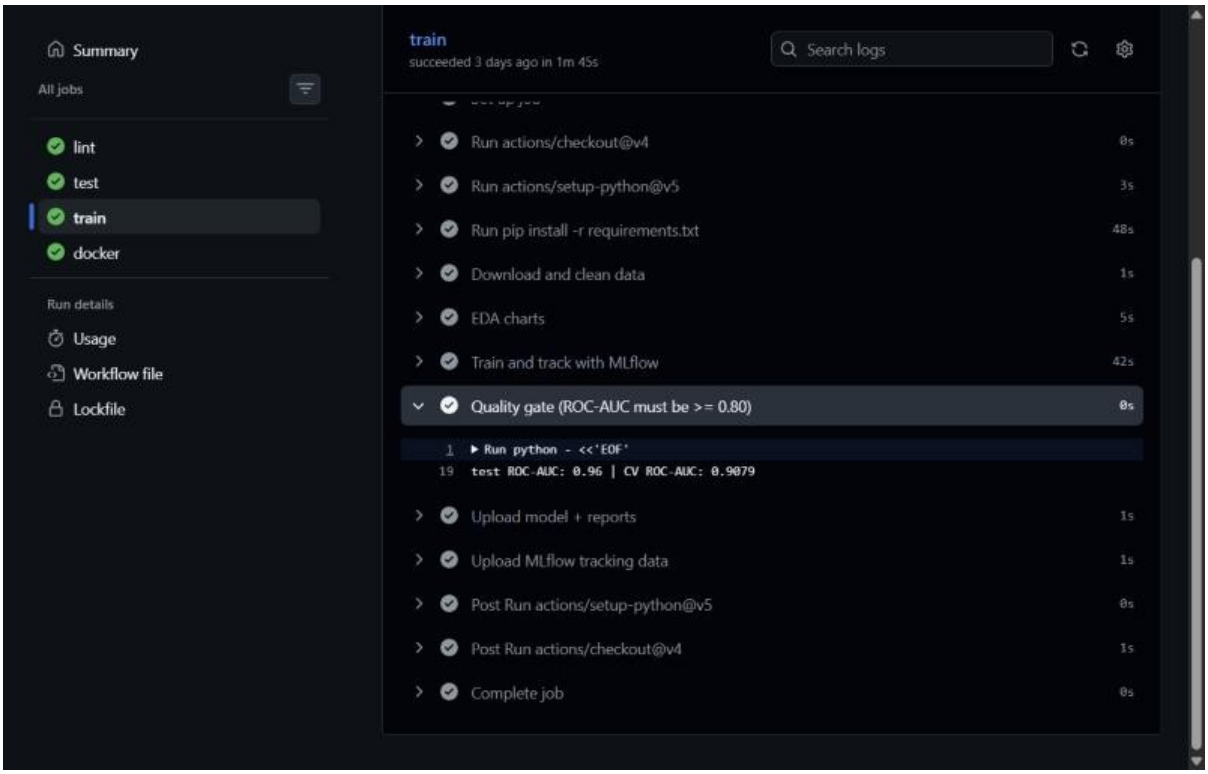


Figure 17. Quality gate step in the train job: test ROC-AUC 0.96, CV ROC-AUC 0.908 (threshold 0.80)

9. Model Containerization

The Dockerfile builds a slim image from python:3.12-slim. It installs only the serving requirements, copies the package, the API and the model, runs as a non-root user and defines a HEALTHCHECK on /health. Dependencies are installed in an early layer so code changes rebuild in seconds.

```
docker build -t heart-disease-api:1.0.0 .
```

```
docker run --rm -p 8000:8000 heart-disease-api:1.0.0

curl -X POST http://localhost:8000/predict -H "Content-Type:
application/json" \
     -d @scripts/sample_request.json

{"prediction":0,"label":"no heart disease","probability_disease":0.2009,
 "confidence":0.7991,"model_version":"1.0.0"}
```

The API validates input with Pydantic: each field has a type and an allowed range or set of codes, and invalid requests get a clear HTTP 422 message. 'confidence' is the probability of the predicted class; 'probability_disease' is always the probability of disease. Interactive documentation is available at /docs.

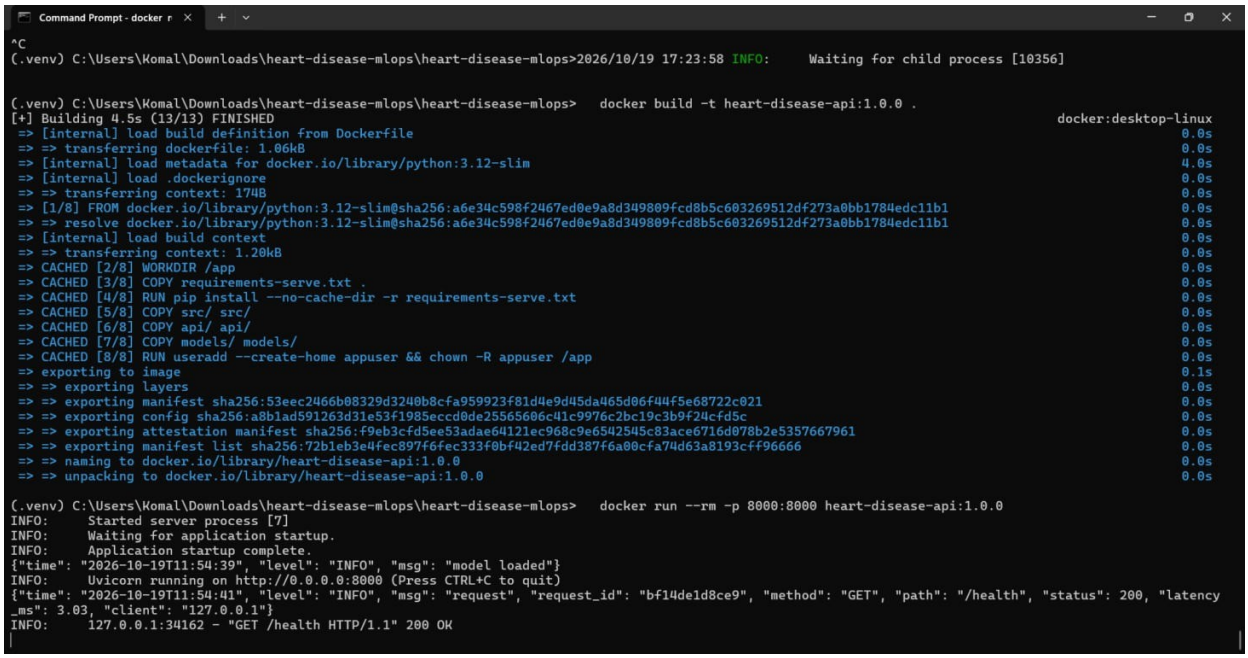


Figure 18. docker build and docker run on the local machine: model loaded, JSON request log for /health

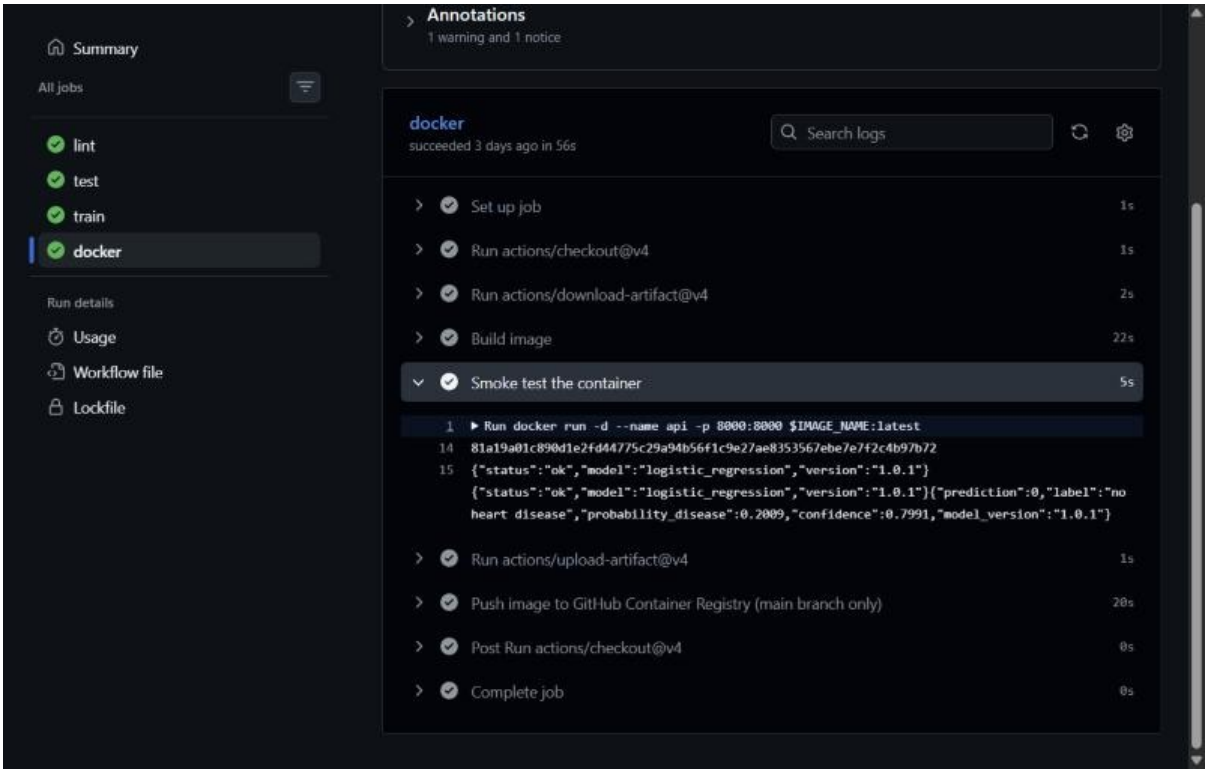


Figure 19. The same image built and smoke-tested in CI (/health and /predict)

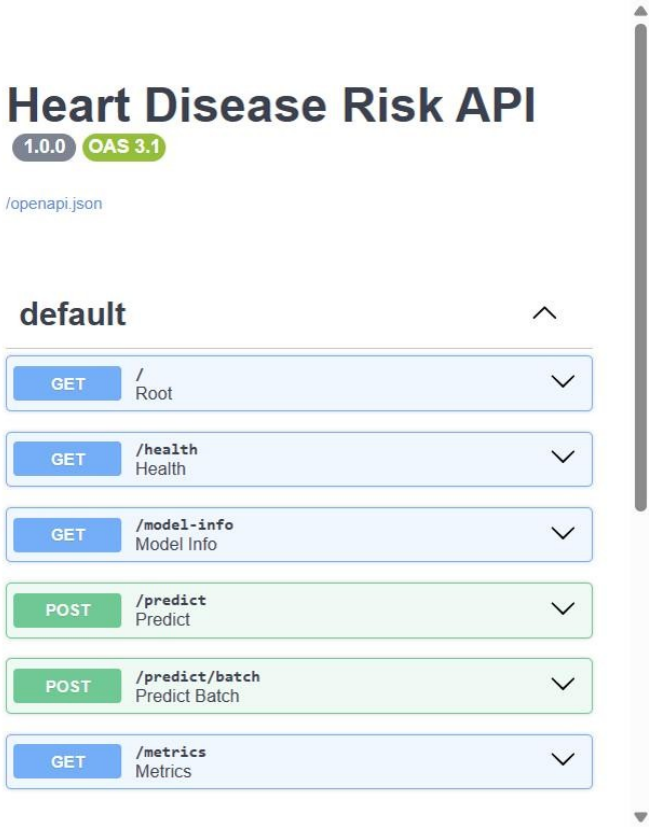


Figure 20. Swagger UI listing the API endpoints

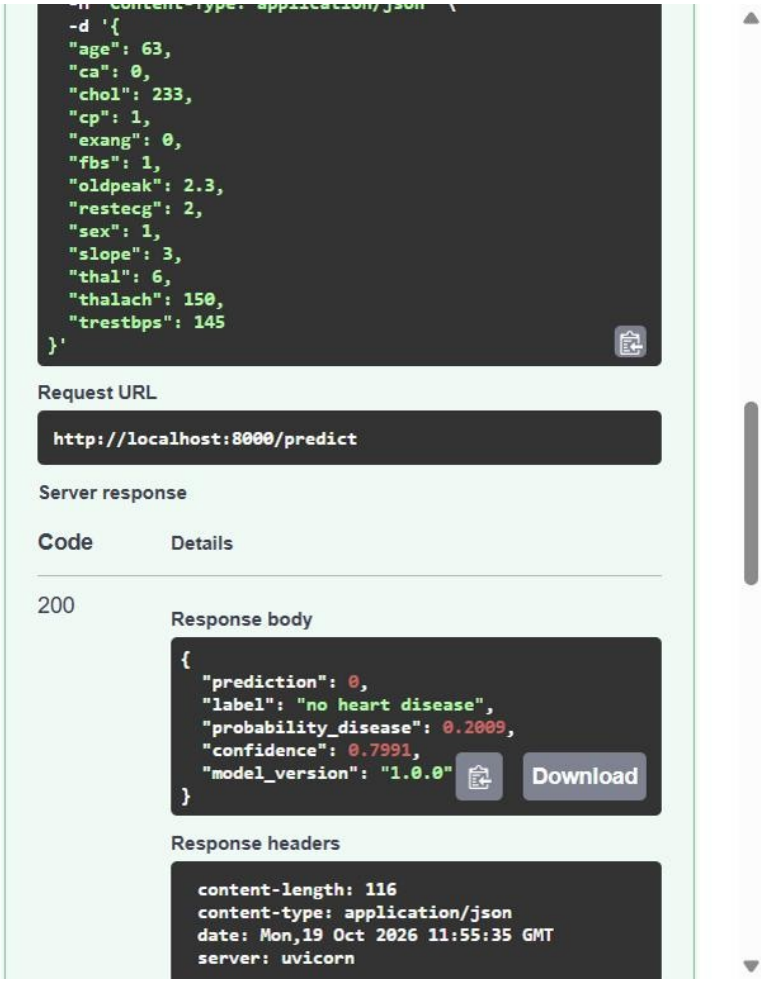


Figure 21. POST /predict on the container: prediction 0 (no heart disease), confidence 0.80

10. Production Deployment on Kubernetes

The API is deployed to a local Kubernetes cluster (Docker Desktop, kind cluster, Kubernetes v1.36) using the manifests in k8s/:

- **Deployment:** 2 replicas, rolling updates with zero unavailable pods, CPU and memory requests/limits, readiness and liveness probes on /health, non-root security context, and Prometheus scrape annotations.
- **Service:** type LoadBalancer on port 80. On Docker Desktop it is reachable on localhost; on Minikube through 'minikube tunnel'; on a cloud cluster it gets a public IP.
- **Optional:** an Ingress (host heart.local) and a HorizontalPodAutoscaler (2 to 5 pods at 70% CPU).

```
kubectl apply -f k8s/00-namespace.yaml -f k8s/01-deployment.yaml -f k8s/02-service.yaml
kubectl -n heart-disease get pods,svc
curl -X POST http://localhost/predict -H "Content-Type: application/json" -d @scripts/sample_request.json
```

```
(.venv) C:\Users\Komal\Downloads\heart-disease-mlops\heart-disease-mlops>kubectl -n heart-disease get pods,svc
NAME                             READY   STATUS    RESTARTS   AGE
pod/heart-api-5b8f8bff7c-8rpbk   1/1     Running   0          3m7s
pod/heart-api-5b8f8bff7c-nb85c   1/1     Running   0          3m18s

NAME                 TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
service/heart-api    LoadBalancer   10.96.75.107   172.18.0.5    80:31451/TCP   2d22h

(.venv) C:\Users\Komal\Downloads\heart-disease-mlops\heart-disease-mlops>kubectl -n heart-disease port-forward svc/heart-api 8080:80
Forwarding from 127.0.0.1:8080 -> 8000
Forwarding from [::1]:8080 -> 8000
```

Figure 22. Two pods Running (1/1) and the LoadBalancer service with external IP 172.18.0.5

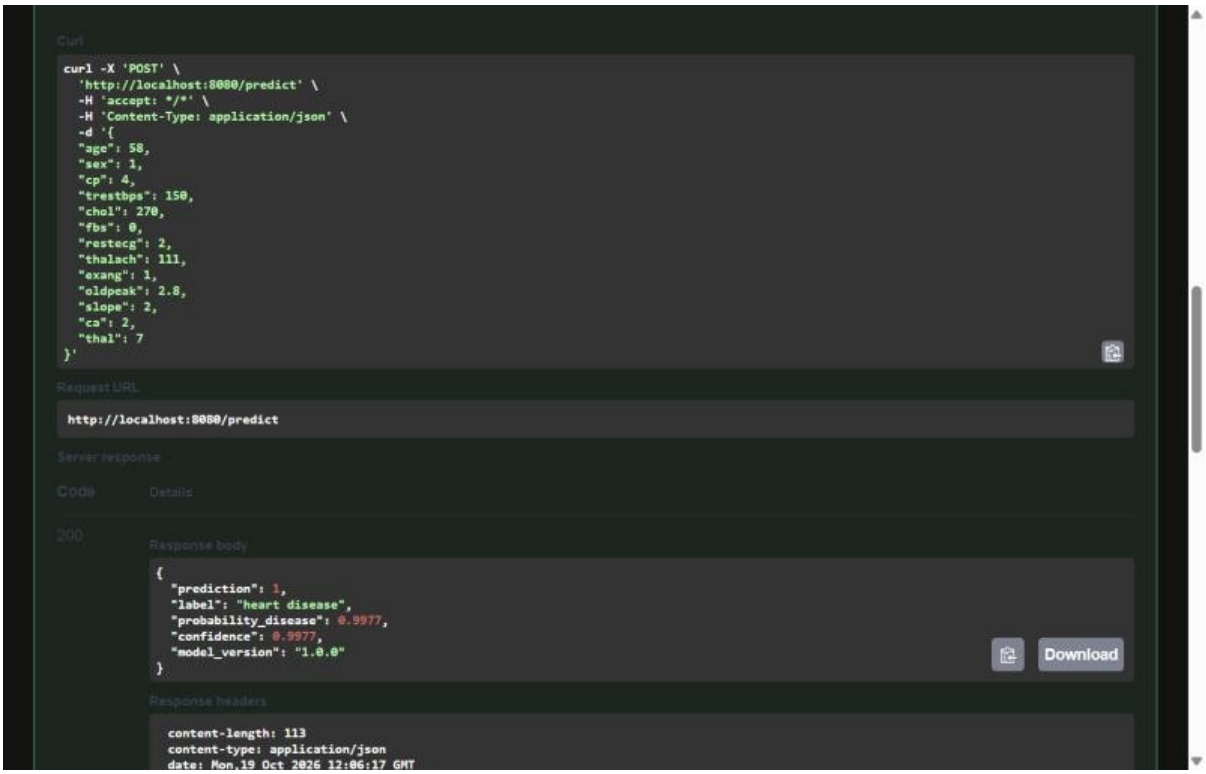


Figure 23. High-risk patient sent through the Kubernetes service: heart disease, confidence 0.998

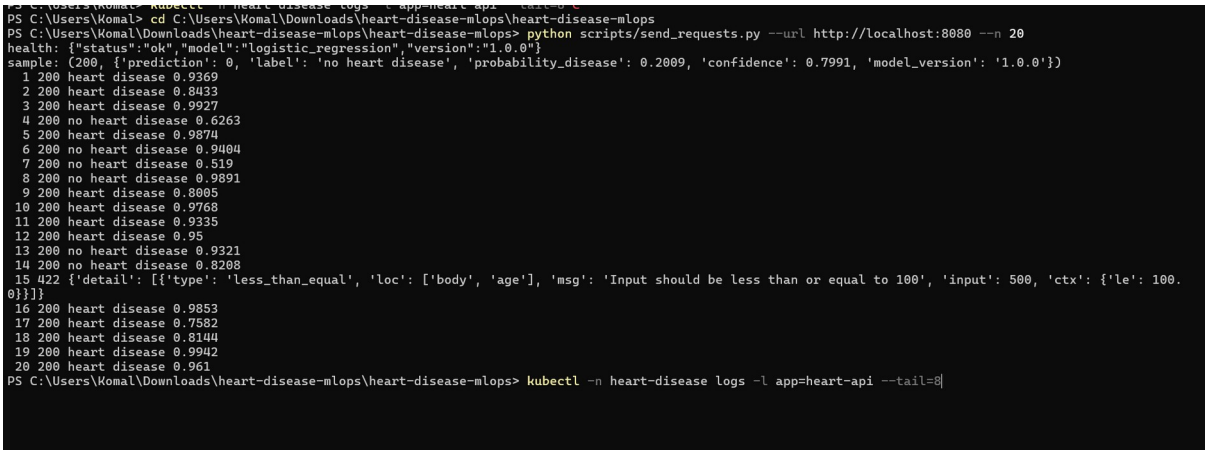


Figure 24. 20 requests through the cluster; request 15 is invalid on purpose and is rejected with HTTP 422

Issue found and fixed. The first rollout failed with CreateContainerConfigError: the pod requires a non-root user, but the image only names its user (appuser), so Kubernetes could not verify it. Adding runAsUser: 1000 to the securityContext fixed it, and both pods became Ready.

11. Monitoring and Logging

Logging. A FastAPI middleware writes one JSON line per request with a request ID, method, path, status code, latency and client address; every prediction is also logged with its result. JSON logs are easy to filter with kubectl logs or to ship to a log system such as Loki or ELK.

Metrics. The /metrics endpoint exposes Prometheus metrics:

Metric	Type	Meaning
api_requests_total	Counter	Requests by method, endpoint and status code
api_request_latency_seconds	Histogram	Request latency per endpoint (p50/p95)

Metric	Type	Meaning
model_predictions_total	Counter	Predictions by class (disease / no disease)
model_prediction_confidence	Histogram	Confidence of predictions (to spot drift)
model_loaded	Gauge	1 when the model is loaded

Dashboard. docker-compose.yml starts the API, Prometheus (5-second scrape) and Grafana with the data source and a dashboard pre-provisioned. The dashboard shows total requests and predictions, error rate, requests per second, p50/p95 latency, predictions by class, average confidence and responses by status code. scripts/send_requests.py generates realistic traffic including a few invalid requests. The same Prometheus and Grafana setup is also provided for Kubernetes in k8s/05-monitoring.yaml, using pod discovery through the scrape annotations.

Watching the share of 'disease' predictions and the average confidence over time is a simple first signal of data drift: if incoming patients change, these numbers move before accuracy can be measured.

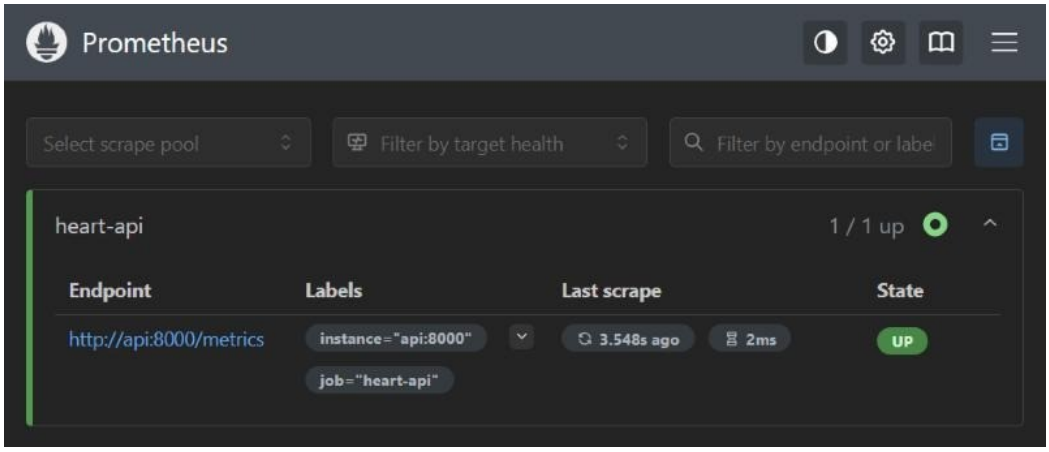


Figure 25. Prometheus scraping the API /metrics endpoint (target UP)

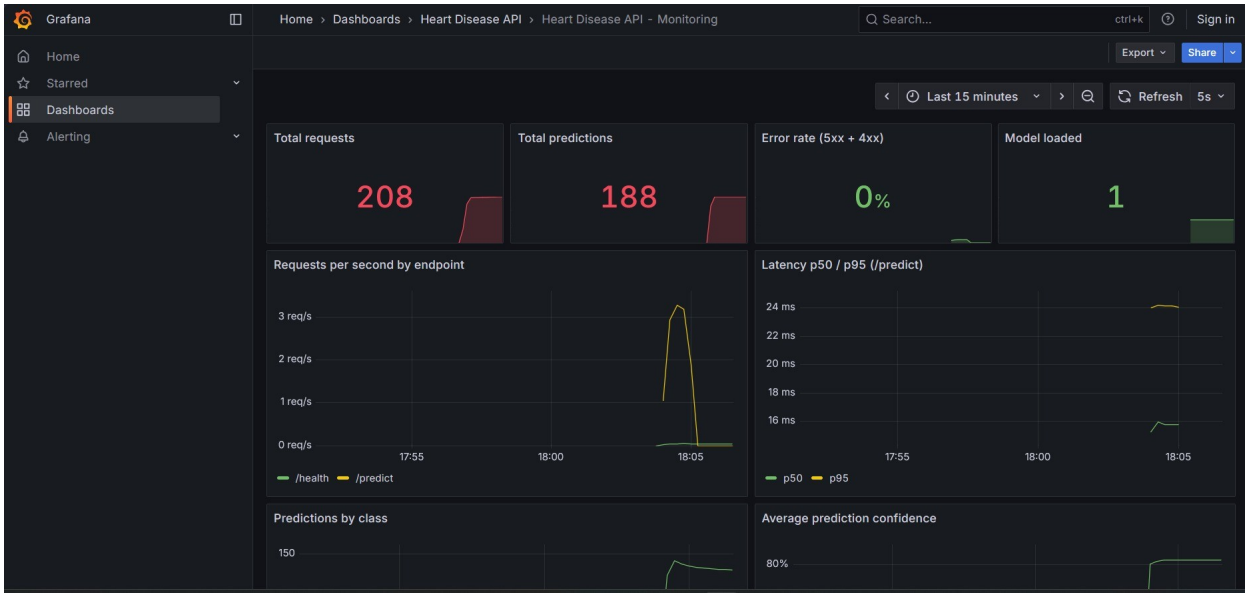


Figure 26. Grafana dashboard after 200 test requests: totals, request rate, p50/p95 latency, predictions by class, confidence

```
PS C:\Users\Komal\Downloads\heart-disease-mlops\heart-disease-mlops> kubectl -n heart-disease logs -l app=heart-api --tail=500 | Select-String '"/predict"|"
prediction"' | Select-Object -Last 10

{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "prediction", "prediction": 1, "label": "heart disease", "probability_disease": 0.9853,
"confidence": 0.9853}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "request", "request_id": "c156f8f9d4b1", "method": "POST", "path": "/predict", "status": 200,
"latency_ms": 12.65, "client": "127.0.0.1"}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "prediction", "prediction": 1, "label": "heart disease", "probability_disease": 0.7582,
"confidence": 0.7582}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "request", "request_id": "61365beb649d", "method": "POST", "path": "/predict", "status": 200,
"latency_ms": 12.22, "client": "127.0.0.1"}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "prediction", "prediction": 1, "label": "heart disease", "probability_disease": 0.8144,
"confidence": 0.8144}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "request", "request_id": "76c274d3923c", "method": "POST", "path": "/predict", "status": 200,
"latency_ms": 10.62, "client": "127.0.0.1"}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "prediction", "prediction": 1, "label": "heart disease", "probability_disease": 0.9942,
"confidence": 0.9942}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "request", "request_id": "d9ab6f392f93", "method": "POST", "path": "/predict", "status": 200,
"latency_ms": 13.65, "client": "127.0.0.1"}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "prediction", "prediction": 1, "label": "heart disease", "probability_disease": 0.961,
"confidence": 0.961}
{"time": "2026-10-19T12:08:31", "level": "INFO", "msg": "request", "request_id": "f67e34da0e6a", "method": "POST", "path": "/predict", "status": 200,
"latency_ms": 9.35, "client": "127.0.0.1"}
```


Figure 27. Structured JSON logs from the pods: one 'prediction' and one 'request' line per call

12. Architecture

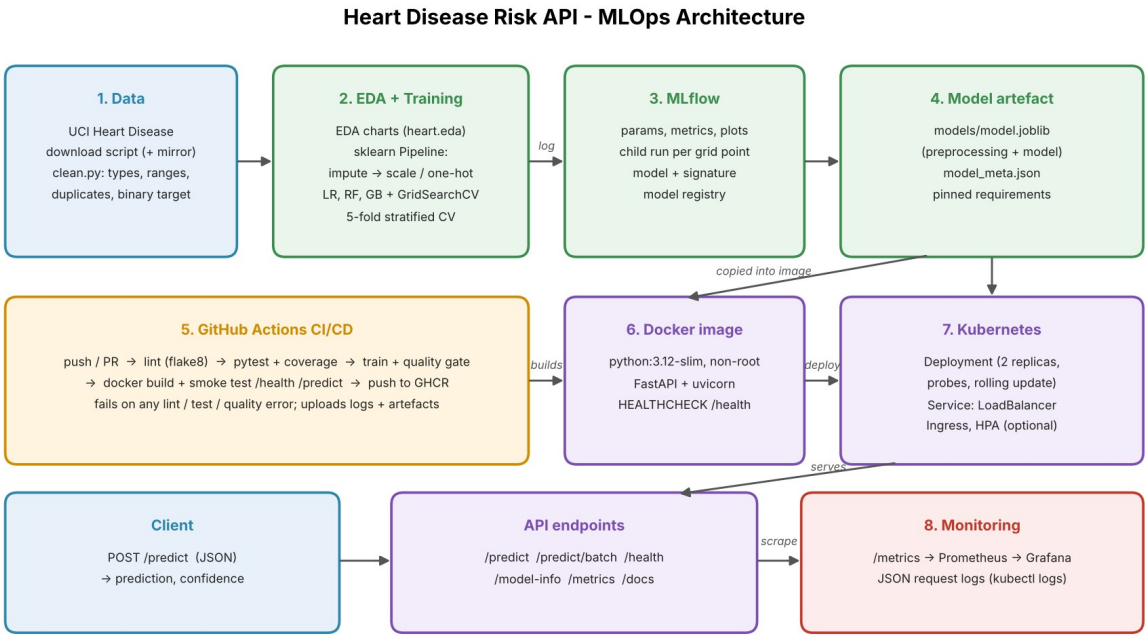


Figure 10. End-to-end architecture

Data flows from the UCI download through cleaning and EDA into training, where every experiment is logged to MLflow and the best pipeline is saved as an artefact. The CI pipeline rebuilds and validates all of this on each push and packages the model into a Docker image, which Kubernetes runs behind a LoadBalancer. Clients call /predict, and Prometheus scrapes /metrics for the Grafana dashboard.

13. Conclusion and Future Work

The project delivers a reproducible, tested and monitored heart disease risk service. The selected Logistic Regression model reaches a test ROC-AUC of 0.96 with high recall, and every step from data download to deployment is automated.

Limitations. The Cleveland dataset is small (303 patients) and from one hospital, so test metrics have wide uncertainty and the model should not be used for real clinical decisions without validation on other populations.

Next steps: add the other UCI sites (Hungary, Switzerland, Long Beach) for external validation; tune the decision threshold for a target recall; add drift detection (e.g. Evidently) and alerting rules; store

MLflow on a remote server with an S3 artefact store; and add continuous deployment that rolls the new image to the cluster after the quality gate passes.

Links

- **Code repository:** https://github.com/Kmatlawala/heart-disease-mlops
- **Pipeline video:** https://github.com/Kmatlawala/heart-disease-mlops/blob/main/report/MLOps_Assignment1_Pipeline_Final.mp4
- **API access:** Local only. Run with docker run -p 8000:8000 heart-disease-api:1.0.0, or in Kubernetes with kubectl -n heart-disease port-forward svc/heart-api 8080:80, then open /docs (see README sections 4 and 5).